

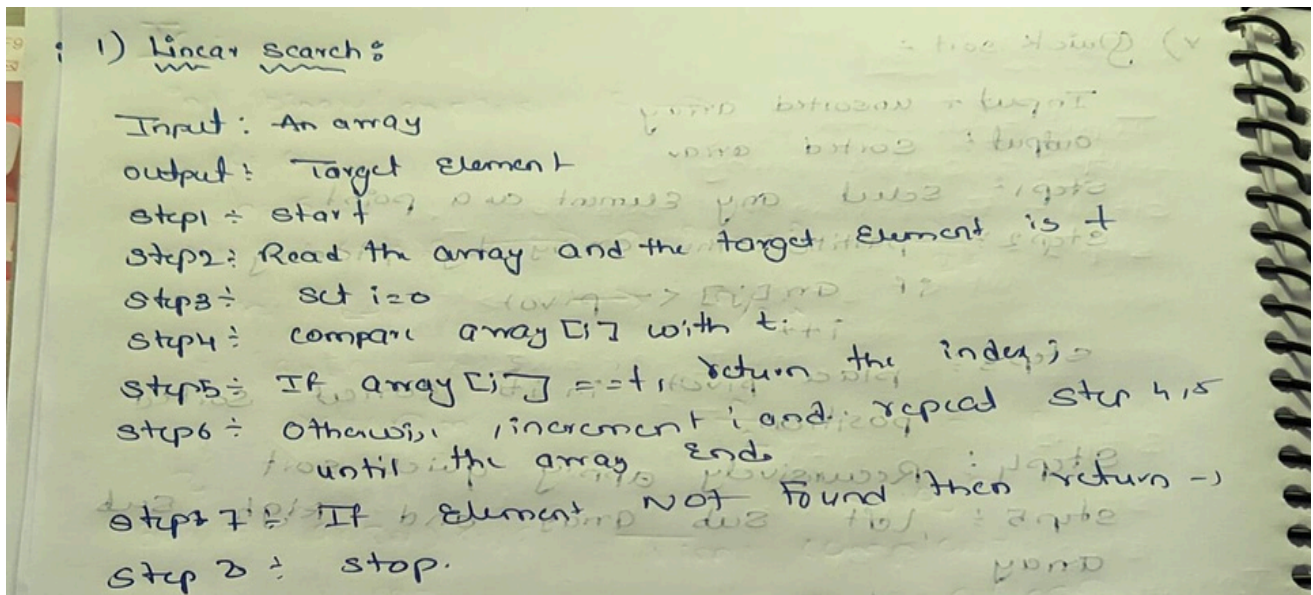
Practical: 2

Implementation and Time analysis of linear and binary search algorithm

05/08/2026

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Algorithm:



Program:

```
#include <iostream>
using namespace std;
int linearsearch(int arr[], int n, int target)
{
    for (int i = 0; i <= n - 1; i++)
    {
        if (arr[i] == target)
        {
            cout << "the target found at index of " << i << " element is " << arr[i] << endl;
        }
    }
    return -1;
}
int main()
{
    int arr[] = {2, 3, 4, 5, 6, 7, 8, 9, 14};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 14;
```

```
linearsearch(arr, n, target);
return 0;
}
```

Output:

```
Sorted array: 1 3 6 10
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ linearsearch.cpp -o linearsearch } ; if ($?) { .\linearsearch }
the target found at index of 8 elemnt is 14
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time Complexity:

Time complexity :

```
for(i=0; i<n; i++) {
    if (arr[i] == target) {
        return i;
    }
}
return -1;
```

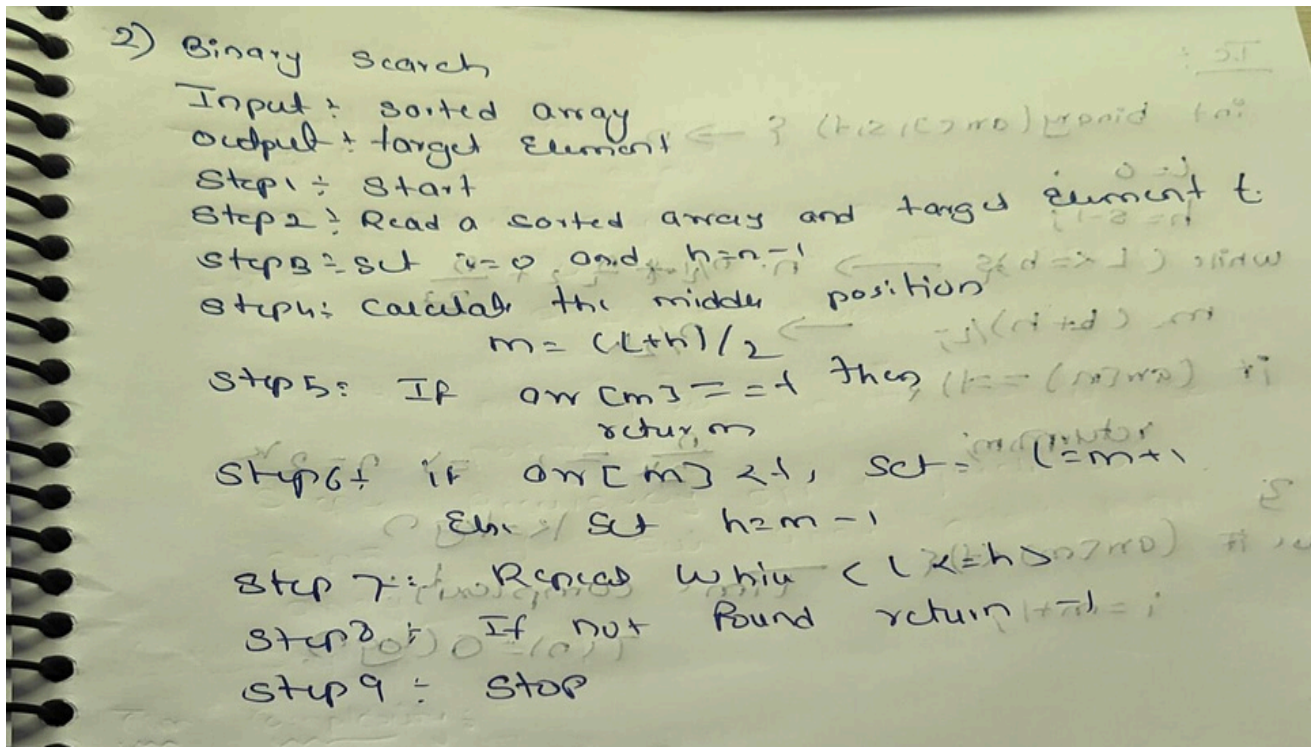
$T(n) \Rightarrow n+1+1+1 \Rightarrow n+3$
 $T(n) = n+3$
 $T(n) = O(n)$

Conclusion:

Hence, From this practical of linear search the algorithm and programs have executed successfully. And Time complexity has categorised into three ways as Best, average and worst.

2. Binary search:

Algorithm:



Program:

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            { // ascending order
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```
void printArray(int arr[], int n)
{
    for(int i=0; i < n; i++) {

        cout<<arr[i] << " ";
        cout<<endl;

    }

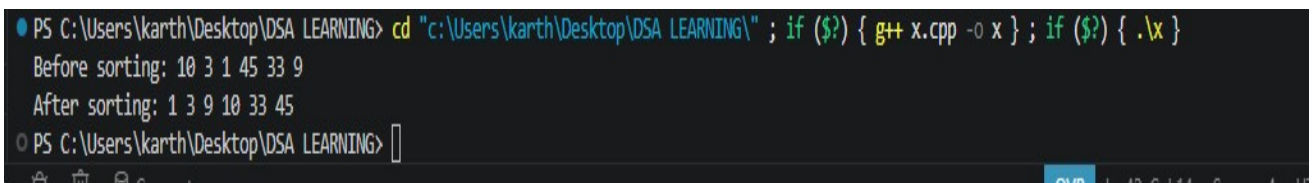
}

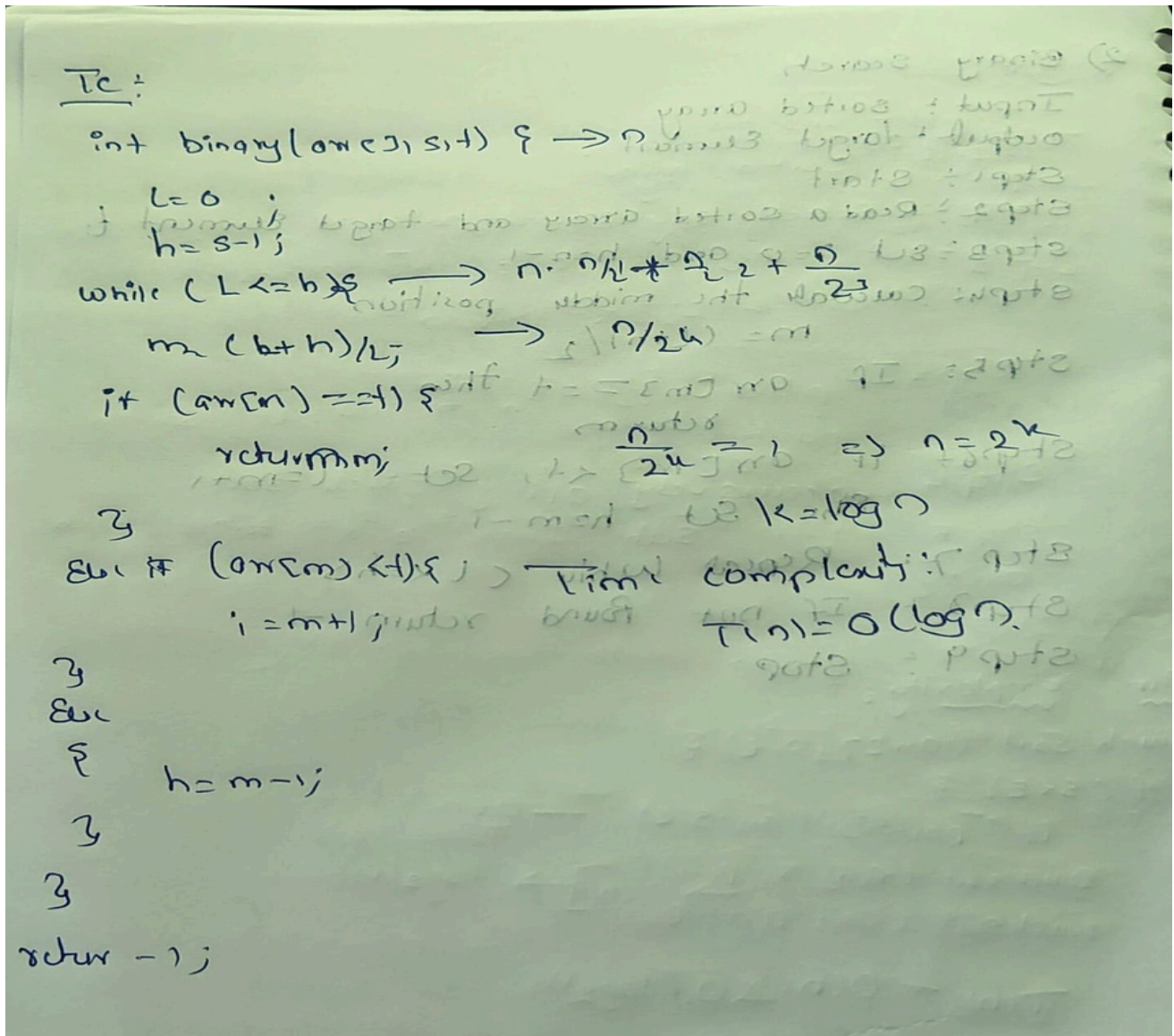
int main()
{
    int arr[]={10, 3, 1, 45, 33, 9};
    int n=sizeof(arr) / sizeof(arr[0]);

    cout<<"Before sorting: ";
    printArray(arr, n);
    bubbleSort(arr, n);

    cout<<"After sorting: ";
    printArray(arr, n);
    return 0;

}
```

Output:***Time Complexity:***



Conclusion:

Hence, From this practical of Binary search the algorithm and programs have executed successfully. And Time complexity has categorised into three ways as Best, average and worst.